

javaSE

hashmap

在JDK1.7和1.8的实现不同；1.7是数组+链表；1.8是数组+链表+红黑树

1.7

根据key计算index，算出hash值后， $index = (n-1) \& hash$ 计算index，将键值对插入数组中下标为index的位置；在插入新元素时，如果计算出的index与之前的相同，发生hash冲突，使用头插法用链表的形式插入到index=1的位置，在JDK1.8后改成尾插法。如果index不同则插入到别的index位置。

1.8

每发生一次hash冲突，链表长度都会+1，当链表长度>8时，先调用treeifybin方法，会根据数组的长度来决定是否将链表转变为红黑树，当数组长度 ≥ 64 的情况下会将链表转为红黑树，如果 < 64 则进行扩容

hashmap扩容

capacity：默认容量是16

loadFactor：负载因子，默认0.75f

扩容后是原来容量的2倍

B+树的特点和优势

关键字数量跟路数相等；

根节点和枝节点上都不会存储数据，只在叶子结点存储数据；

每个叶子节点增加了一个指向相邻叶子结点的指针，形成一个有序链表的结构。

优势

- 1)它是 B Tree 的变种, B Tree 能解决的问题, 它都能解决。B Tree 解决的两大问题是 (每个节点存储更多关键字; 路数更多)
- 2)扫库、扫表能力更强 (如果我们要对表进行全表扫描, 只需要遍历叶子节点就可以了, 不需要遍历整棵 B+Tree 拿到所有的数据)
- 3) B+Tree 的磁盘读写能力相对于 B Tree 来说更强, 同数据量下磁盘I/O次数更少 (根节点和枝节点不保存数据区, 所以一个节点可以保存更多的关键字, 一次磁盘加载的关键字更多)
- 4)范围查询和排序能力更强 (因为叶子节点上有下一个数据区的指针, 数据形成了链表)
- 5)效率更加稳定 (B+Tree 永远是在叶子节点拿到数据, 所以 IO 次数是稳定的)

优先队列

保证每次从队列中取出的元素都是权值最小的

通过完全二叉树实现的小顶堆 (任意一个非叶子结点的权值, 都不大于其左右子节点的权值)

红黑树和最小堆

红黑树插入最差要比较 $2\log N$, 最小堆最坏情况是 $\log N$

红黑树删除不需要比较, 只需要不超过3次旋转, 而最小堆需要 $2\log N$ 次

最小堆最大的优势在于取最小值性能是 $O(1)$,如果插入的数据基本有序, 那么最小堆能获得比较好的性能, 在频繁不断地取最小值的是否满足条件的应用中有更加好的性能 (如超时队列), 红黑树相应就要尽量避免不必要的查询次数才能在超时队列这种应用中获得更好的性能

JAVAE

操作系统

多线程

实现多线程有几种方法

- 1) 继承Thread类

(1) 定义Thread类的子类，并重写该类的run方法，该run方法的方法体就代表了线程要完成的任务。因此把run()方法称为执行体。

(2) 创建Thread子类的实例，即创建了线程对象。

(3) 调用线程对象的start()方法来启动该线程。

```
public class MyThread extends Thread {  
    public void run() {  
        System.out.println("MyThread.run()");  
    }  
}  
  
MyThread myThread1 = new MyThread();  
myThread1.start();
```

2) 实现Runnable接口

(1) 定义Runnable接口的实现类，并重写该接口的run()方法，该run()方法的方法体同样是该线程的线程执行体。

(2) 创建 Runnable实现类的实例，并依此实例作为Thread的target来创建Thread对象，该Thread对象才是真正的线程对象。

(3) 调用线程对象的start()方法来启动该线程

```
public class MyThread extends OtherClass implements Runnable {  
    public void run() {  
        System.out.println("MyThread.run()");  
    }  
}
```

3) 实现Callable接口

1) 创建Callable接口的实现类，并实现call()方法，该call()方法将作为线程执行体，并且有返回值。

(2) 创建Callable实现类的实例，使用FutureTask类来包装Callable对象，该FutureTask对象封装了该Callable对象的call()方法的返回值。

(3) 使用FutureTask对象作为Thread对象的target创建并启动新线程。

(4) 调用FutureTask对象的get()方法来获得子线程执行结束后的返回值

```
public class SomeCallable extends OtherClass implements Callable {
```

```
    @Override
    public V call() throws Exception {
        .... // TODO Auto-generated method stub
        .... return null;
    }
}
```

```
Callable oneCallable = new SomeCallable();
```

```
//由Callable创建一个FutureTask对象:
```

```
FutureTask oneTask = new FutureTask(oneCallable);
```

```
//注释: FutureTask是一个包装器, 它通过接受Callable来创建, 它同时实现了Future和
Runnable接口。
```

```
//由FutureTask创建一个Thread对象:
```

```
Thread oneThread = new Thread(oneTask);
```

```
oneThread.start();
```

```
//至此, 一个线程就创建完成了。
```

线程上下文切换

多线程的上下文切换是指 CPU 控制权由一个已经正在运行的线程切换到另外一个就绪并等待获取 CPU 执行权的线程的过程。

可能有以下原因:

线程的 cpu 时间片用完(每个线程轮流执行, 看前面并发的概念)

垃圾回收

有更高优先级的线程需要运行

线程自己调用了 sleep、yield、wait、join、park、synchronized、lock 等方法

当 Context Switch 发生时, 需要由操作系统保存当前线程的状态, 并恢复另一个线程的状态

框架

spring框架的初步理解

IOC和AOP

IoC:

Inversion of Control 控制反转。我们以前开发，在一个类中使用其他类对象的时候都是采用new的方式直接获取，或者高级一点是通过反射的方式得到需要的对象实例。这就造成了程序的耦合度非常高，一个类的运行，严重依赖于其他的类。并且还会出现程序中硬编码的情况。而spring中的IoC很好的解决了该问题，我们在一个类中使用其他类对象时，只需要定义一个接口类型的类成员变量，由使用者在使用时为我们注入具体的实现类对象，从而降低了程序的耦合度。

它是spring的核心之一。或者说是spring的基础核心，spring的其余核心功能都必须有IoC的支持。

控制反转指的是，我们在获取对象的时候，由之前的主动出击，变成了被动接收。也就是说，在编写某个类时，只需要提供一个接口类型的类成员，并不需要关系具体的实现类，而是由使用者在使用时提供。这就降低了类和类之间的耦合度。

实现IoC的思想就只有两种：依赖注入（Dependency Injection，简称DI）和依赖查找（Dependency Lookup）。

而依赖注入使用的更广泛一些。例如：构造函数注入,set方法注入等等。

首先，明确依赖注入是实现控制反转的一种思想（另一种是依赖查找）。

其次，在开发过程中，我们需要某个类的实例时，是由使用者为我们提供该类的实例。而不是自己去获取。

最后，实现依赖注入的方式可以是使用构造方法注入或者set方法注入两种方式。

在spring中注入的方式就有很多了，比如constructor-arg元素，property元素，p名称空间等等。

在持久层、业务层和控制层分别采用 @Repository、@Service 和 @Controller 对分层中的类进行注释，而用 @Component 对那些比较中立的类进行注释

AOP:

Aspect Oriented Programming 面向切面编程。

我们之前学习的java语言，号称是面向对象编程，它有自己的优势。但是也存在着一些弊端。举例说明，在实际开发中，我们都会有一根业务主线，即客户（甲方）的需求。而程序员要做的就是围绕业务主线上的需求，实现功能（实现功能的方法我们叫做业务核心方法）。但是不可避免的，会有一些功能与业务主线没有关系，却又不能不做，比如权限的控制，事务的控制，日志的记录等等，这些功能绝大多数时候和业务主线没有关系，但是却和很多业务核心方法交织在一起，使我们的开发变得麻烦，并且冗余代码增多。而spring的提供了一种思想，把这些和业务主线没有关系的功能剥离出来，而在需要使用这些公共方法时，适时适地的把它加到我们的代码中去，使程序员在开发时，把更多的精力放在理解需求，实现业务核心功能上，并且让我们的代码变得简洁。这种思想就是面向切面编程。

Mybatis框架的初步理解

SqlSessionFactory对象 创建SqlSession的工厂对象 连接数据库

SqlSession对象 以mybatis的方式操作数据库-内部整合JDBC连接

编程步骤

- 1.创建SqlSessionFactory
- 2.通过SqlSessionFactory创建SqlSession
- 3.通过sqlsession执行数据库操作
- 4.调用session.commit()提交事物
- 5.调用session.close()关闭会话

mybatis的一级缓存和二级缓存:

Mybatis首先缓存中查询结果集,如果没有则查数据库,如果有则从缓存中取出返回结果集就不走数据库.Mybatis内部缓存使用HashMap, key为HashCode+sqlId+Sql语句.Value为从查询出来映射生成的Java对象Mybatis的二级缓存即查询缓存,它的作用域是一个mapper的namespace,记在同一个那么space中查询sql可以缓存中获取数据.二级缓存可以跨SqlSession的

算法

排序算法稳定性

只有冒泡、归并、插入排序是稳定的，其他都不稳定